Library Management System

Spring Boot CRUD Application

Entities: Authors & Books

Submitted by: Kushal Talati

Course: Database Assignment

Date: May 2026

1. Project Overview

This project is a **Spring Boot 2.7 web application** that manages a library database with two core entities: **Author** and **Book**. It demonstrates full Create, Read, and Update (CRUD) operations through a layered MVC architecture using **JPA/Hibernate** for persistence, **JSP/JSTL** for the view layer, and an **H2 in-memory database** seeded with 10 rows per table on startup.

Technology	Version	Purpose
Spring Boot	2.7.18	Application framework
Spring MVC	5.3.x	HTTP controllers + view binding
Spring Data JPA	2.7.x	Repository layer / ORM
Hibernate	5.6.x	JPA implementation
H2 Database	2.1.x	In-memory relational DB
JSP / JSTL	1.2	Server-side view rendering
JUnit 5 + Mockito	5.x / 4.x	Unit testing
Java	11	Runtime
Maven	3.9.6	Build tool

2. Entity Relationship Design

The application models a classic library domain. An **Author** can write many **Books**, establishing a **One-to-Many** relationship. From the Book side this is a **Many-to-One** relationship back to Author.

AUTHORS table		BOOKS table
PK id (BIGINT)	1 ■■■■■■■■ N	PK id (BIGINT)
name (VARCHAR)		title (VARCHAR)
email (VARCHAR UNIQUE)		genre (VARCHAR)
nationality (VARCHAR)		published_year (INT)
birth_year (INT)		isbn (VARCHAR UNIQUE)
		FK author_id → authors.id

2.1 Author Entity

```
@Entity
@Table(name = "authors")
public class Author {
@Id @GeneratedValue(strategy = GenerationType.IDENTITY)
private Long id;

@NotBlank private String name;
@NotBlank @Email @Column(unique = true) private String email;
@NotBlank private String nationality;
@NotNull @Column(name = "birth_year") private Integer birthYear;

@OneToMany(mappedBy = "author", cascade = CascadeType.ALL, orphanRemoval = true)
private List<Book> books = new ArrayList<>();
}
```

2.2 Book Entity

```
@Entity
@Table(name = "books")
public class Book {
@Id @GeneratedValue(strategy = GenerationType.IDENTITY)
private Long id;

@NotBlank private String title;
@NotBlank private String genre;
@NotNull @Column(name = "published_year") private Integer publishedYear;
@NotBlank @Column(unique = true) private String isbn;

@NotNull
@ManyToOne(fetch = FetchType.EAGER)
@JoinColumn(name = "author_id", nullable = false)
private Author author;
}
```

3. Populate Database (10 rows each)

On application startup the **DataInitializer** component (implementing CommandLineRunner) seeds the H2 database with 10 Authors and 10 Books if the tables are empty. JPA creates the schema automatically via spring.jpa.hibernate.ddl-auto=create-drop.

```
@Component
public class DataInitializer implements CommandLineRunner {
@Override
public void run(String... args) {
if (authorRepository.count() > 0) return;

Author a1 = authorRepository.save(new Author("George Orwell", "orwell@example.com",
"British", 1903));
Author a2 = authorRepository.save(new Author("J.K. Rowling", "rowling@example.com",
"British", 1965));
Author a3 = authorRepository.save(new Author("F. Scott
Fitzgerald","fitzgerald@example.com","American", 1896));
// ... 7 more authors

bookRepository.save(new Book("Nineteen Eighty-Four", "Dystopian", 1949, "978-0451524935",
a1));
bookRepository.save(new Book("Animal Farm", "Political", 1945, "978-0451526342", a1));
bookRepository.save(new Book("Harry Potter and the Sorcerer's Stone","Fantasy", 1997,
"978-0590353427", a2));
// ... 7 more books
}
}
```

4. Read Operation

4.1 Custom Inner Join Query (Repository Layer)

The **BookRepository** defines a JPQL constructor-expression query that performs an **INNER JOIN** between the books and authors tables and maps the result directly into a **BookWithAuthorDTO**.

```
// BookRepository.java
@Query("SELECT new com.library.dto.BookWithAuthorDTO(" +
"b.id, b.title, b.genre, b.publishedYear, b.isbn, " +
"a.id, a.name, a.nationality) " +
"FROM Book b INNER JOIN b.author a")
List<BookWithAuthorDTO> findAllBooksWithAuthorDetails();
```

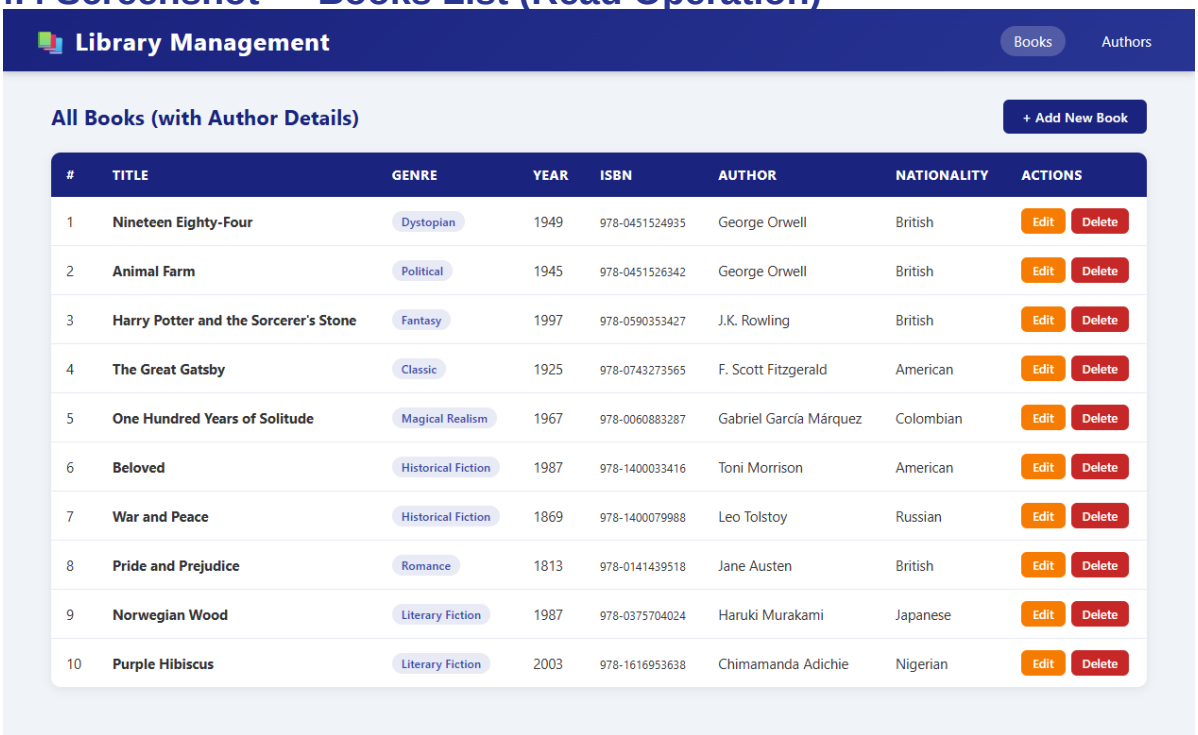
4.2 Service Layer

```
// BookServiceImpl.java
@Override
@Transactional(readOnly = true)
public List<BookWithAuthorDTO> getAllBooksWithAuthorDetails() {
return bookRepository.findAllBooksWithAuthorDetails();
}
```

4.3 Controller

```
// BookController.java
@GetMapping
public String listBooks(Model model) {
model.addAttribute("bookDetails", bookService.getAllBooksWithAuthorDetails());
return "books/list";
}
```

4.4 Screenshot — Books List (Read Operation)



#	TITLE	GENRE	YEAR	ISBN	AUTHOR	NATIONALITY	ACTIONS
1	Nineteen Eighty-Four	Dystopian	1949	978-0451524935	George Orwell	British	Edit Delete
2	Animal Farm	Political	1945	978-0451526342	George Orwell	British	Edit Delete
3	Harry Potter and the Sorcerer's Stone	Fantasy	1997	978-0590353427	J.K. Rowling	British	Edit Delete
4	The Great Gatsby	Classic	1925	978-0743273565	F. Scott Fitzgerald	American	Edit Delete
5	One Hundred Years of Solitude	Magical Realism	1967	978-0060883287	Gabriel García Márquez	Colombian	Edit Delete
6	Beloved	Historical Fiction	1987	978-1400033416	Toni Morrison	American	Edit Delete
7	War and Peace	Historical Fiction	1869	978-1400079988	Leo Tolstoy	Russian	Edit Delete
8	Pride and Prejudice	Romance	1813	978-0141439518	Jane Austen	British	Edit Delete
9	Norwegian Wood	Literary Fiction	1987	978-0375704024	Haruki Murakami	Japanese	Edit Delete
10	Purple Hibiscus	Literary Fiction	2003	978-1616953638	Chimamanda Adichie	Nigerian	Edit Delete

Fig 1: Books list view showing inner-join result — book details + author info in one table

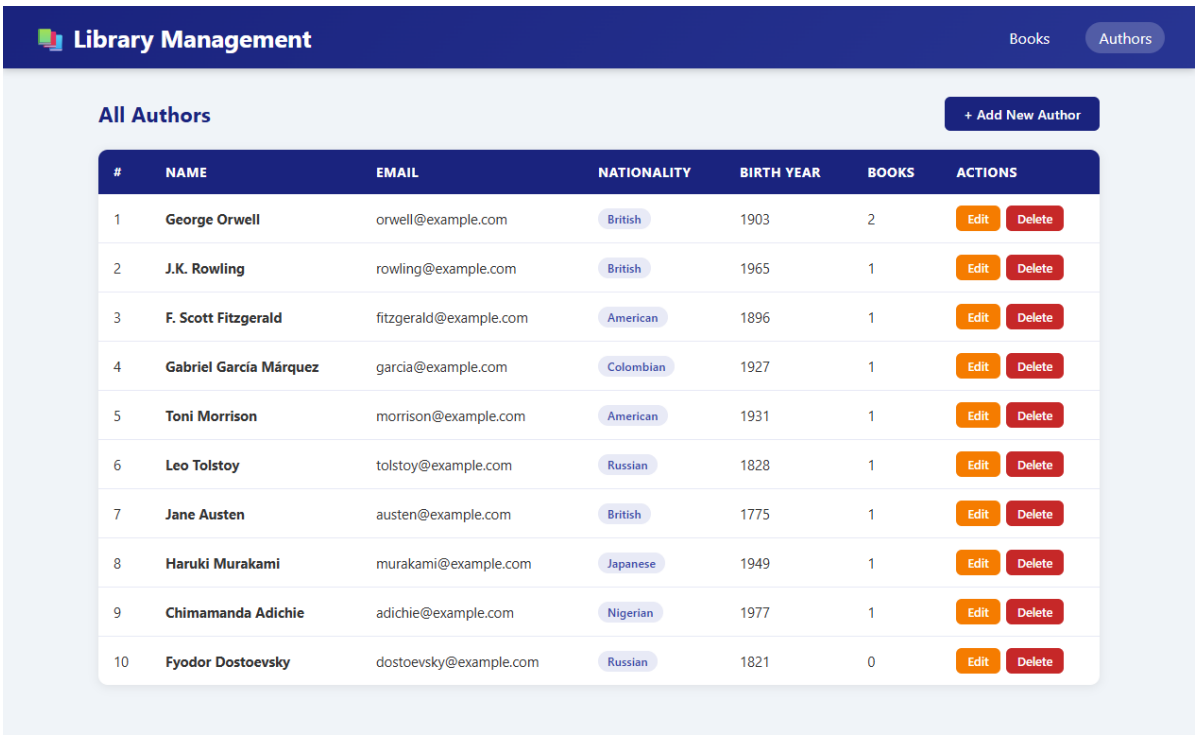


Fig 2: Authors list view

5. Create Operation

5.1 Service Layer — Duplicate Validation

```
// BookServiceImpl.java
@Override
public Book saveBook(Book book) {
if (bookRepository.findByIsbn(book.getIsbn()).isPresent()) {
throw new DataIntegrityViolationException(
"A book with ISBN '" + book.getIsbn() + "' already exists.");
}
return bookRepository.save(book);
}
```

5.2 Controller

```
// BookController.java
@PostMapping("/add")
public String addBook(@Valid @ModelAttribute("book") Book book,
BindingResult result,
@RequestParam("authorId") Long authorId,
Model model, RedirectAttributes redirectAttributes) {
if (result.hasErrors()) {
model.addAttribute("authors", authorService.getAllAuthors());
return "books/add";
}
try {
Author author = authorService.getAuthorById(authorId)
.orElseThrow(() -> new IllegalArgumentException("Author not found"));
book.setAuthor(author);
bookService.saveBook(book);
redirectAttributes.addFlashAttribute("successMessage", "Book added successfully!");
return "redirect:/books";
} catch (DataIntegrityViolationException e) {
result.rejectValue("isbn", "duplicate.isbn", e.getMessage());
model.addAttribute("authors", authorService.getAllAuthors());
return "books/add";
}
}
```

5.3 Screenshots — Add Book

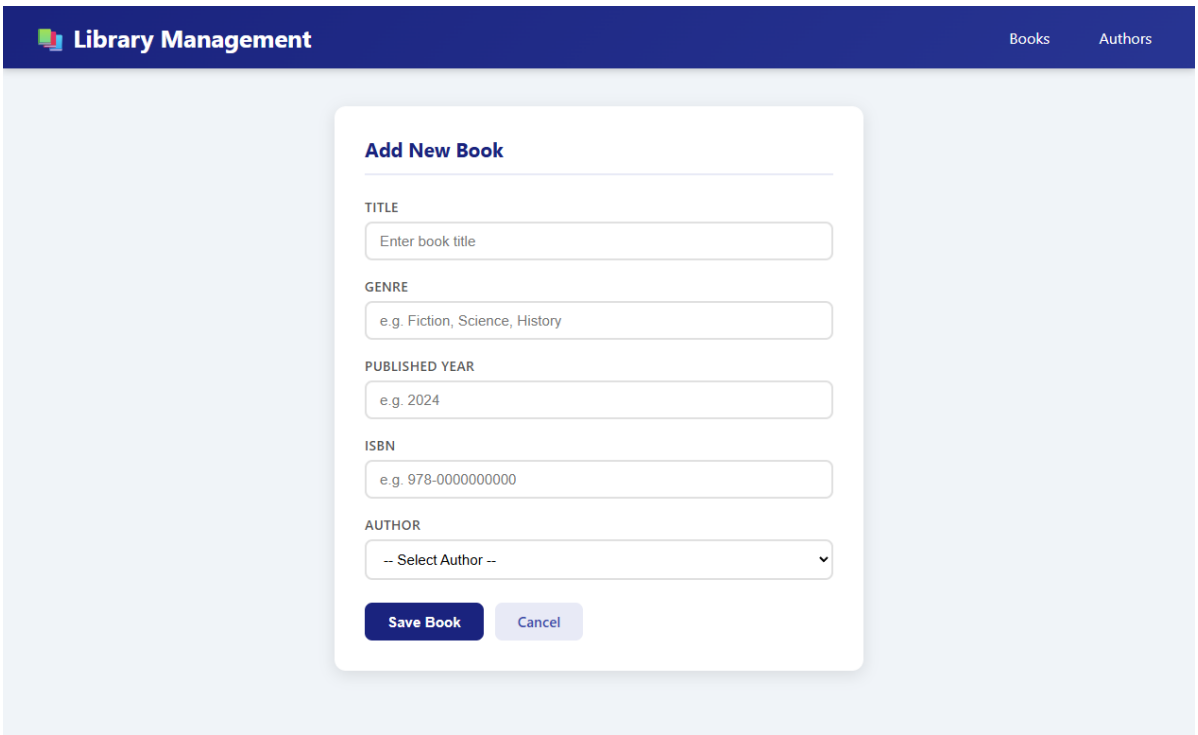


Fig 3: Add Book form (blank)

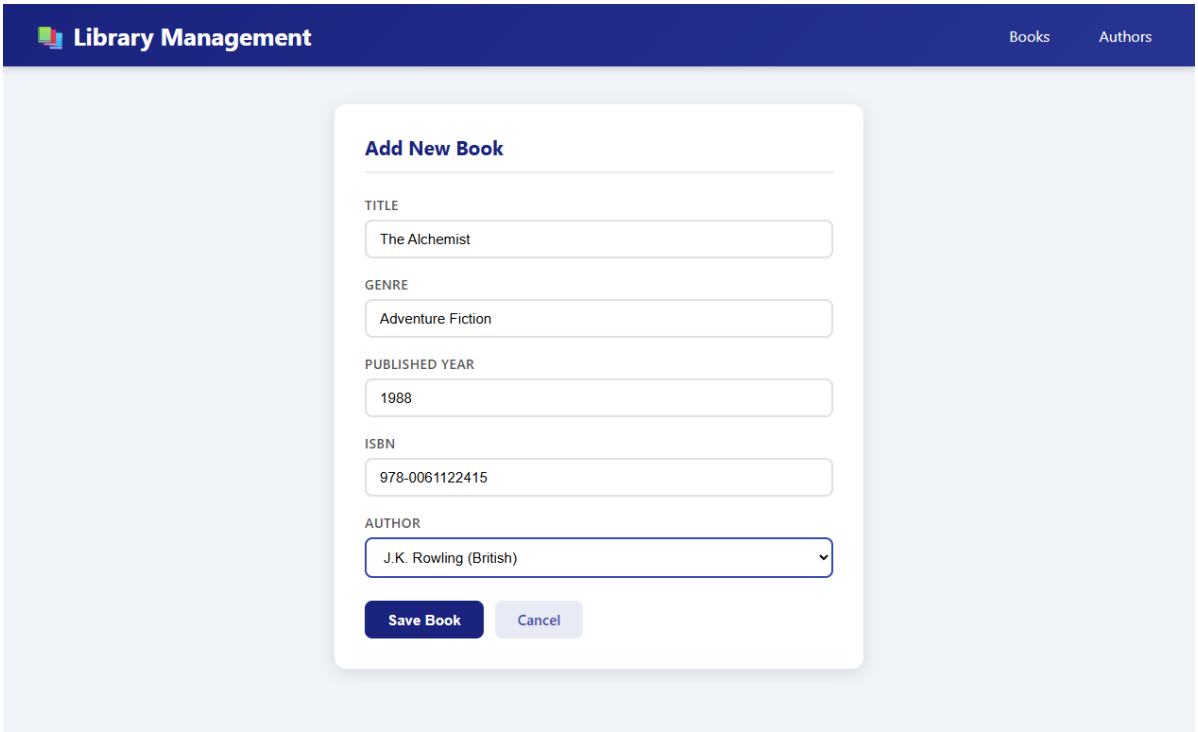


Fig 4: Add Book form — filled in with new book details

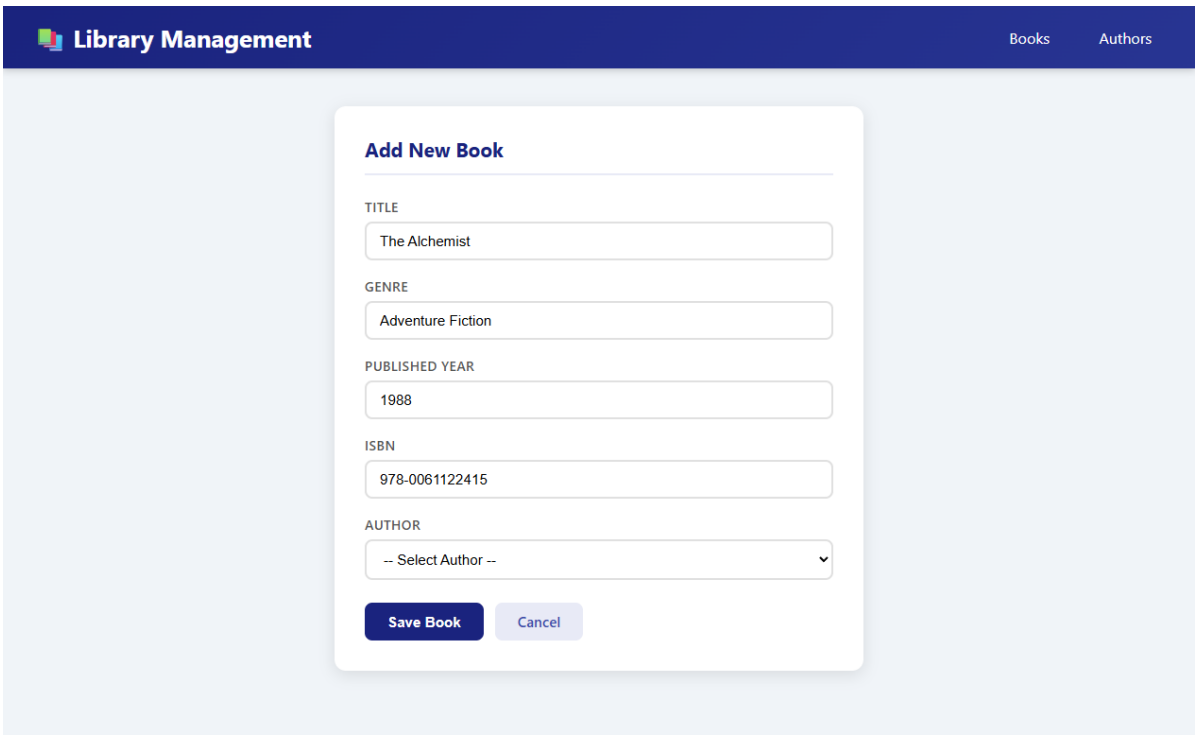


Fig 5: Books list after successful add — success message shown

5.4 Screenshots — Add Author

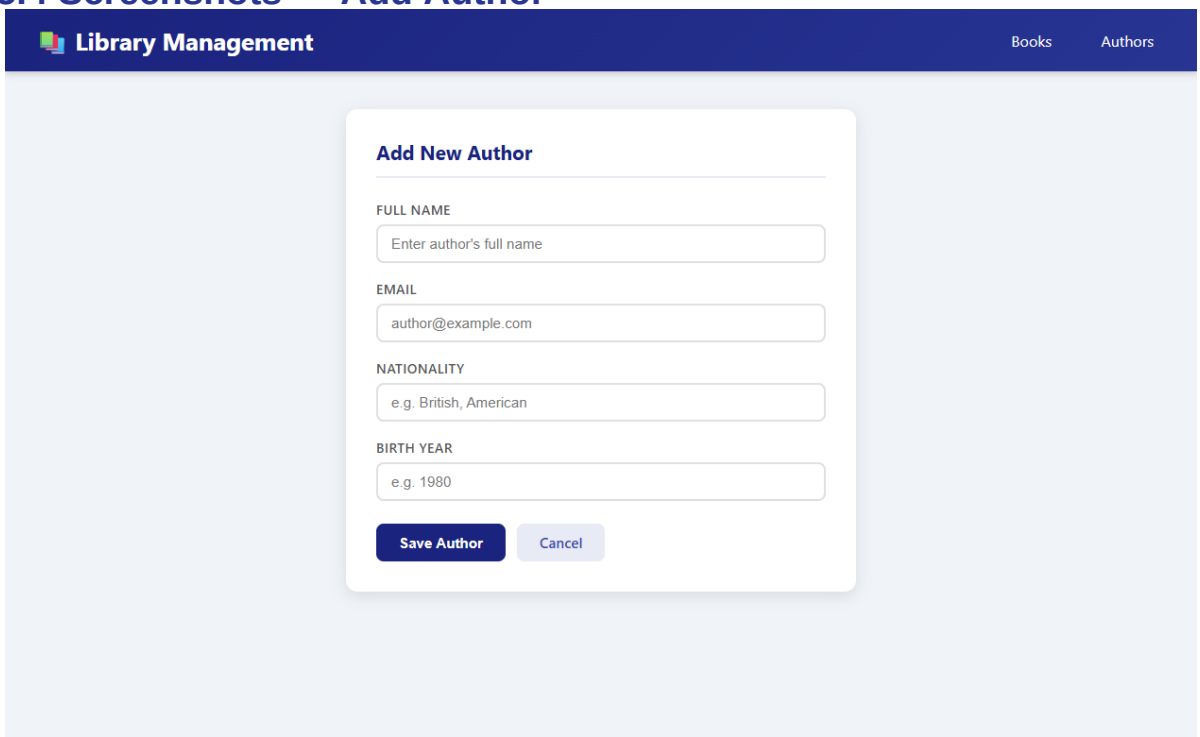


Fig 6: Add Author form (blank)

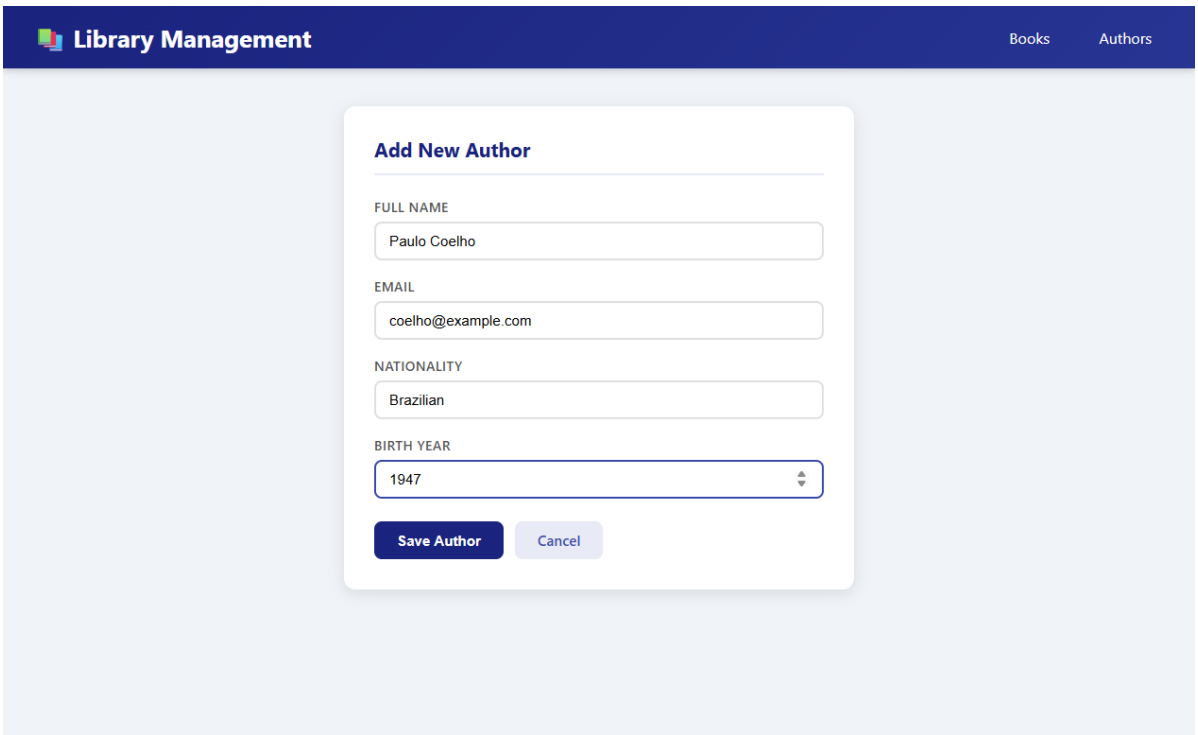


Fig 7: Add Author form — filled in

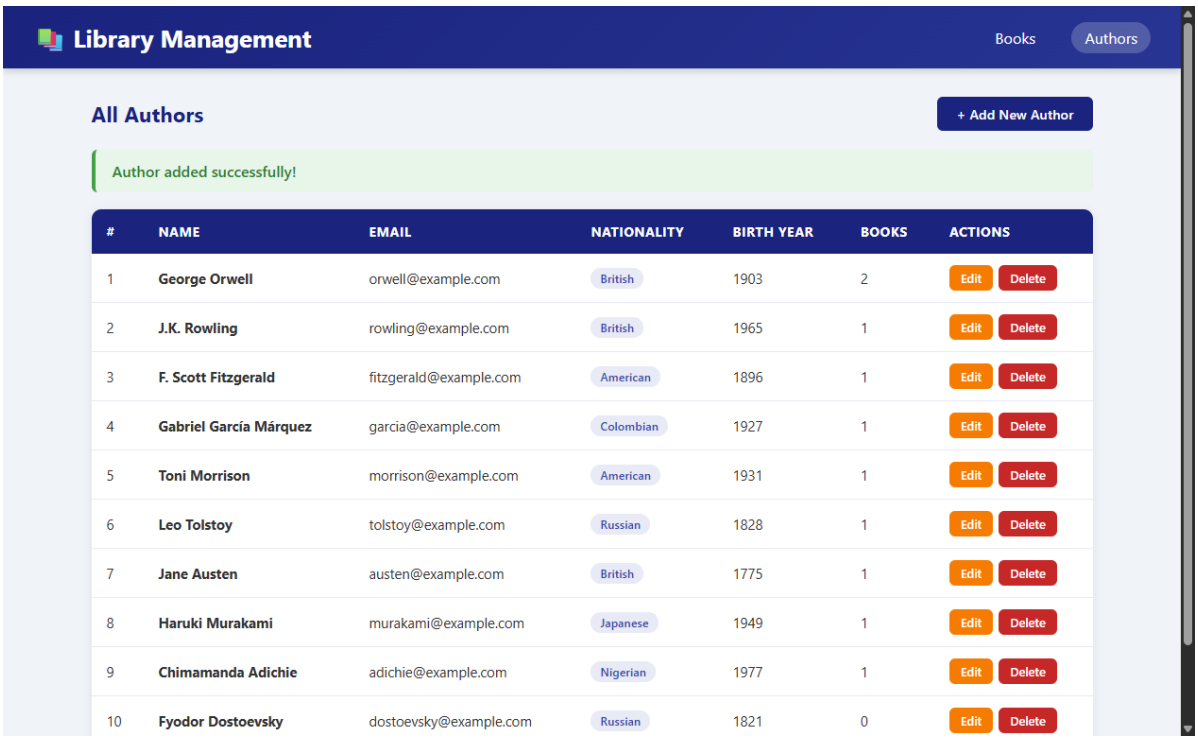


Fig 8: Authors list after successful add

5.5 Exception Handling — Duplicate ISBN / Email

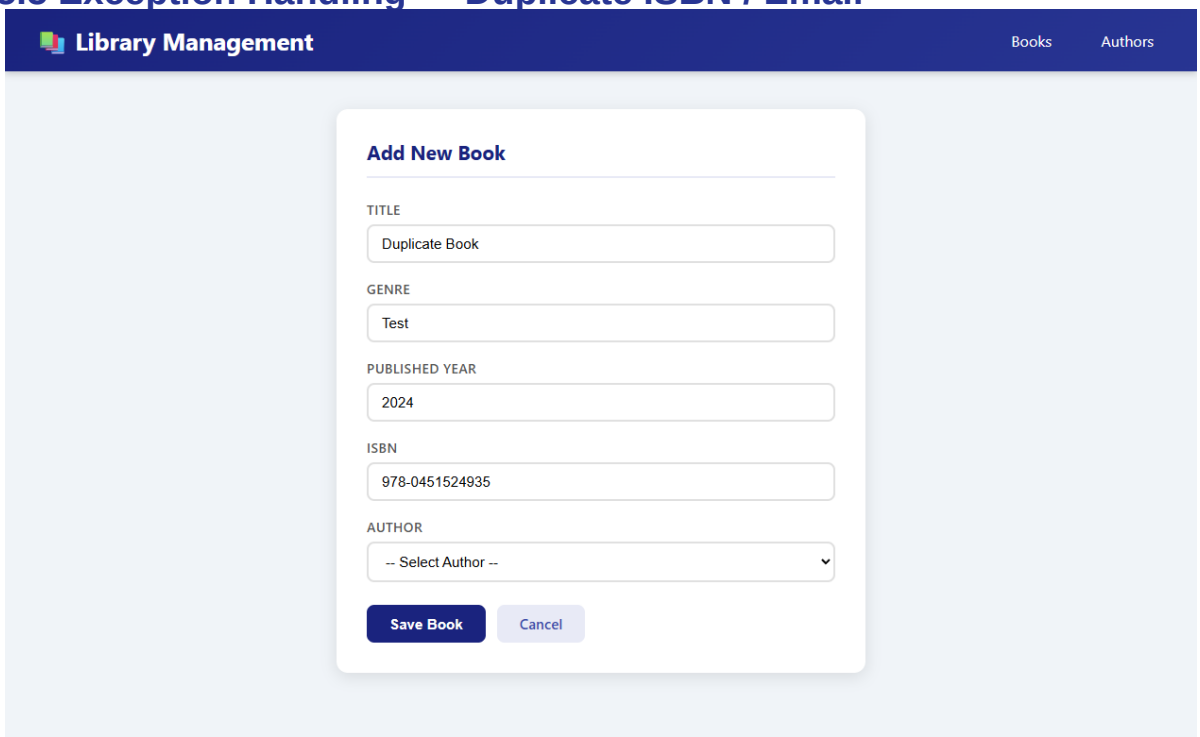


Fig 9: Inline error shown when duplicate ISBN is submitted

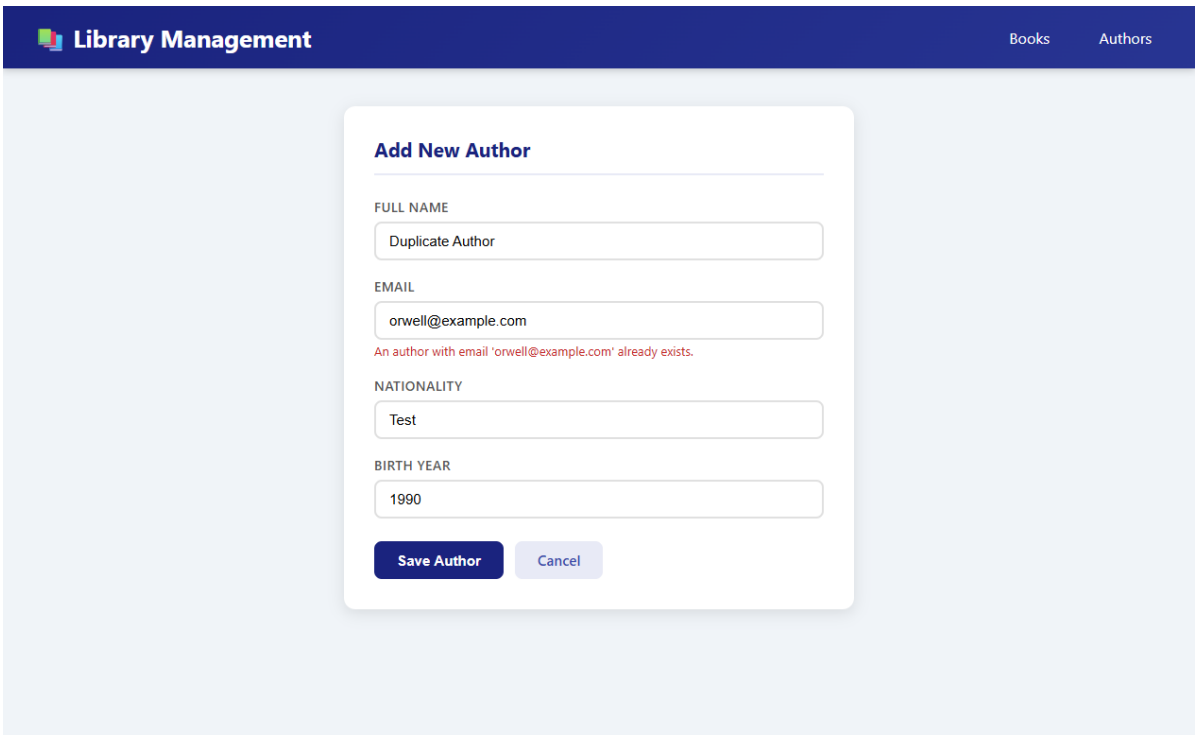


Fig 10: Inline error shown when duplicate Email is submitted

6. Update Operation

6.1 Service Layer

```
// BookServiceImpl.java
@Override
public Book updateBook(Long id, Book updatedBook) {
Book existing = bookRepository.findById(id)
.orElseThrow(() -> new IllegalArgumentException("Book not found with id: " + id));

Optional<Book> isbnConflict = bookRepository.findByIsbn(updatedBook.getIsbn());
if (isbnConflict.isPresent() && !isbnConflict.get().getId().equals(id)) {
throw new DataIntegrityViolationException(
"ISBN '" + updatedBook.getIsbn() + "' is already used by another book.");
}

existing.setTitle(updatedBook.getTitle());
existing.setGenre(updatedBook.getGenre());
existing.setPublishedYear(updatedBook.getPublishedYear());
existing.setIsbn(updatedBook.getIsbn());
existing.setAuthor(updatedBook.getAuthor());
return bookRepository.save(existing);
}
```

6.2 Controller

```
// BookController.java
@PostMapping("/edit/{id}")
public String updateBook(@PathVariable Long id,
@Valid @ModelAttribute("book") Book book,
BindingResult result,
@RequestParam("authorId") Long authorId,
Model model, RedirectAttributes redirectAttributes) {
if (result.hasErrors()) {
model.addAttribute("authors", authorService.getAllAuthors());
return "books/edit";
}
try {
Author author = authorService.getAuthorById(authorId).orElseThrow(
() -> new IllegalArgumentException("Author not found"));
book.setAuthor(author);
bookService.updateBook(id, book);
redirectAttributes.addFlashAttribute("successMessage", "Book updated successfully!");
return "redirect:/books";
} catch (DataIntegrityViolationException e) {
result.rejectValue("isbn", "duplicate.isbn", e.getMessage());
model.addAttribute("authors", authorService.getAllAuthors());
return "books/edit";
}
}
```

6.3 Screenshots — Update Book

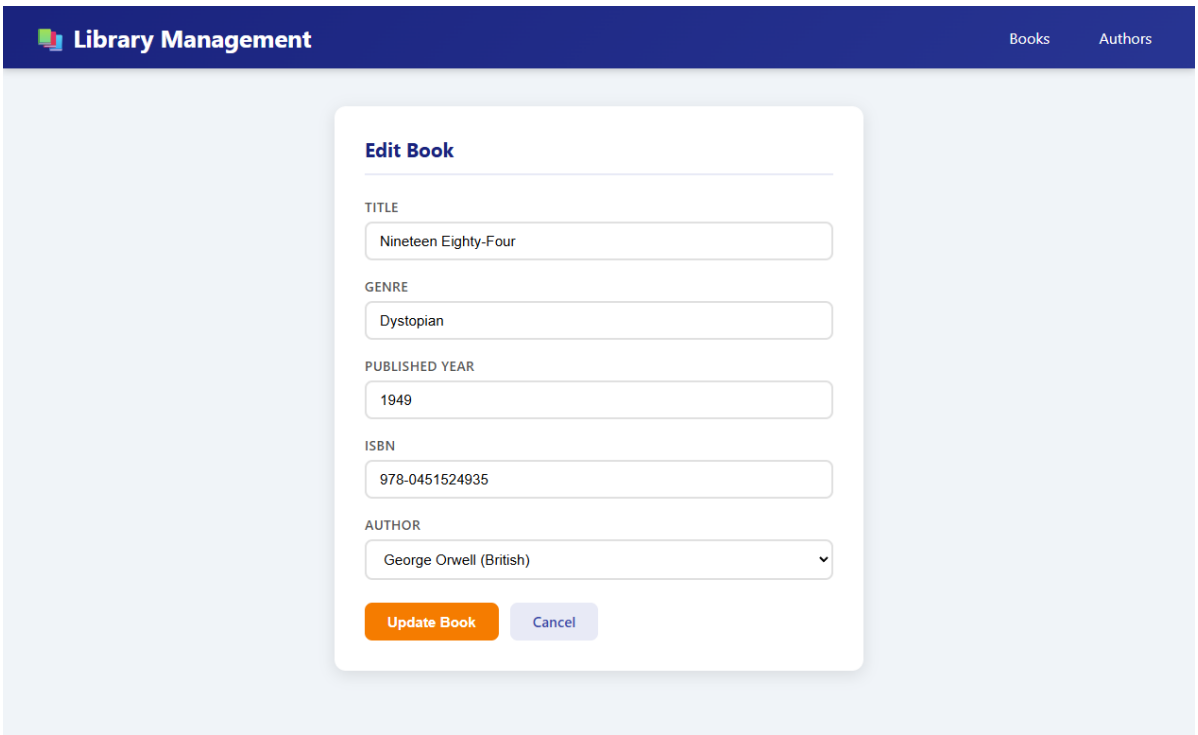


Fig 11: Edit Book form — pre-populated with existing data

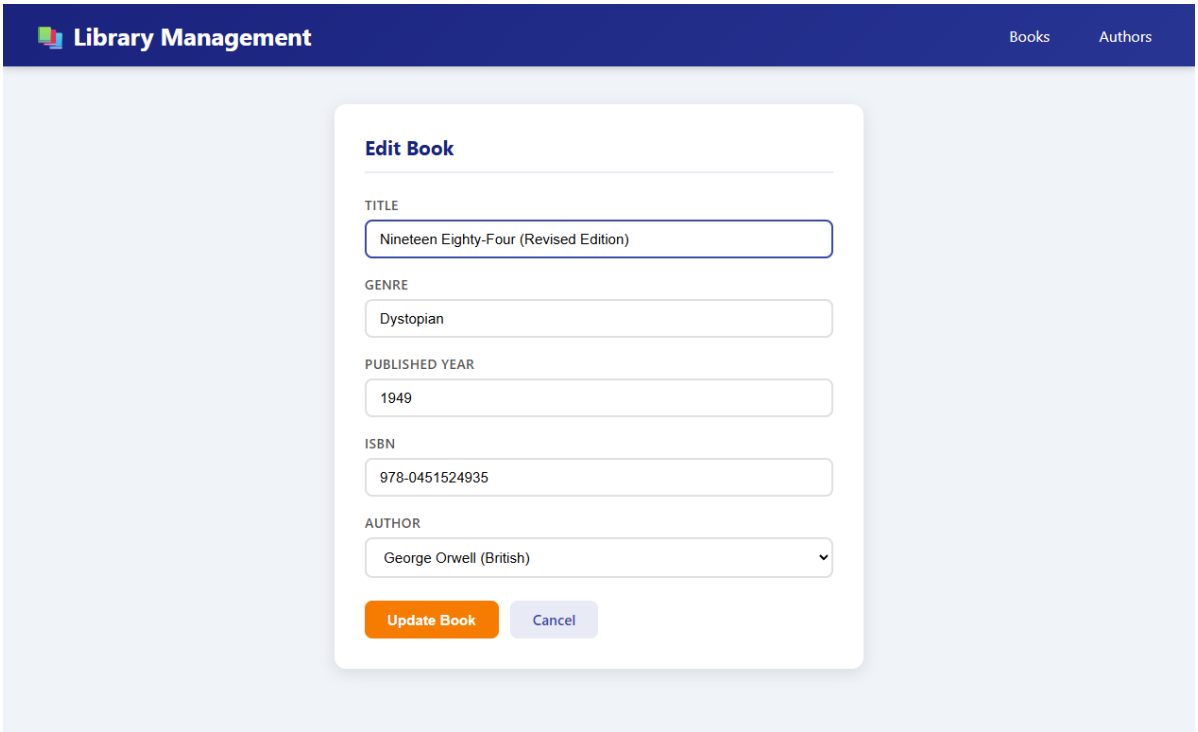


Fig 12: Edit Book form — title modified

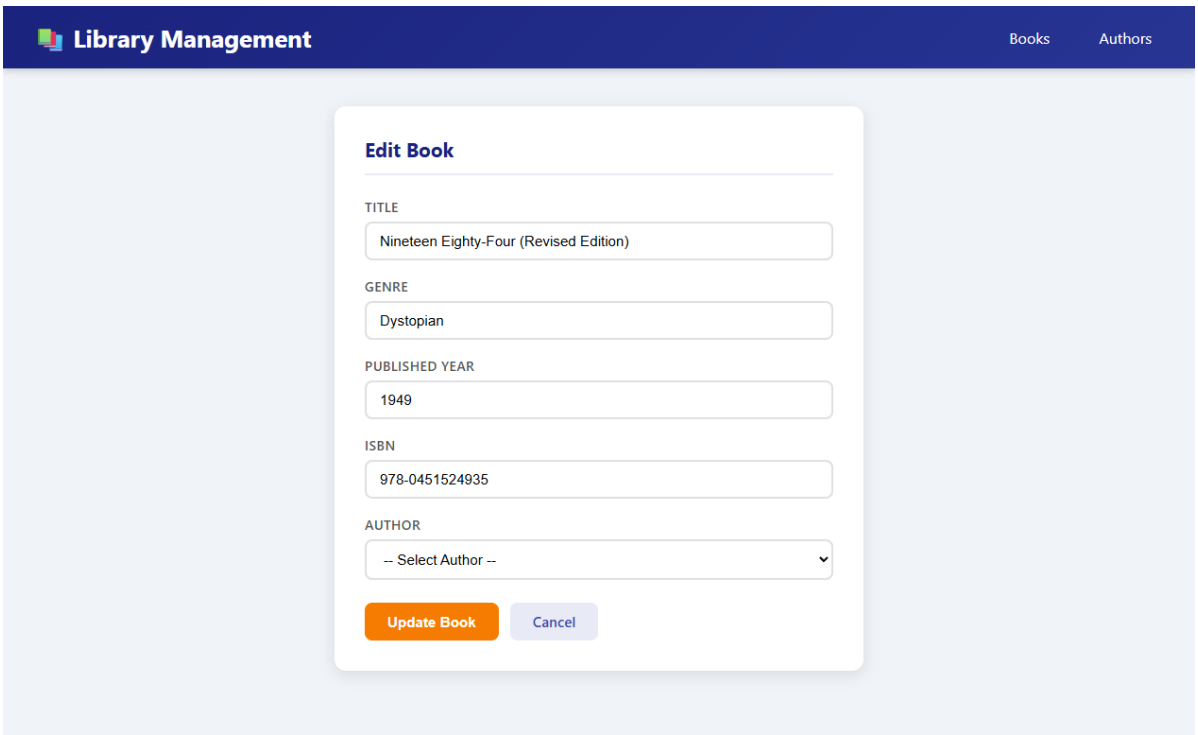


Fig 13: Books list after successful update — success message shown

6.4 Screenshots — Update Author

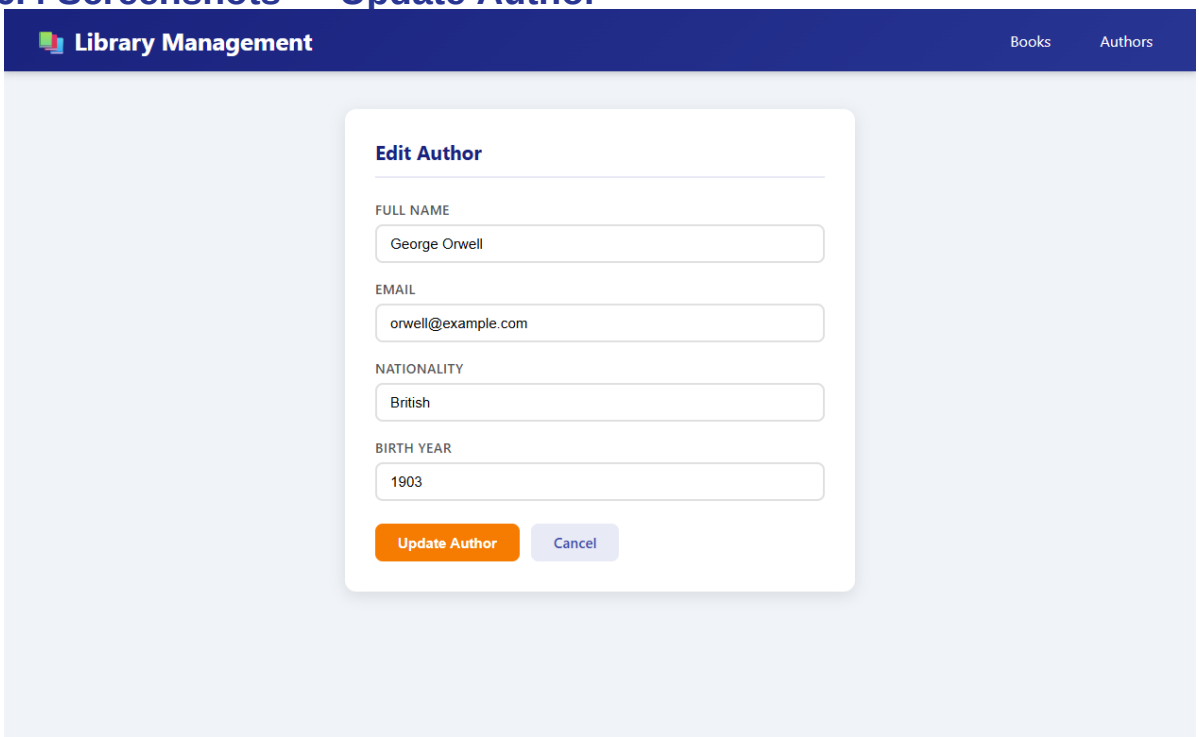


Fig 14: Edit Author form — pre-populated

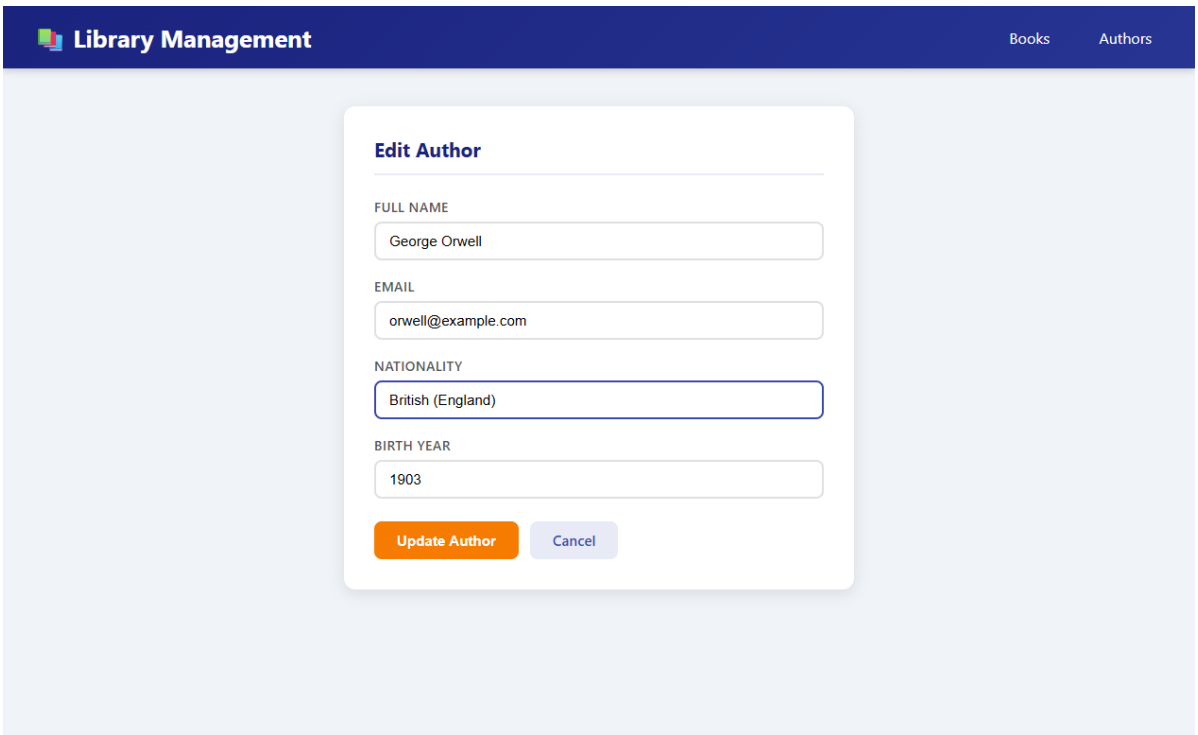
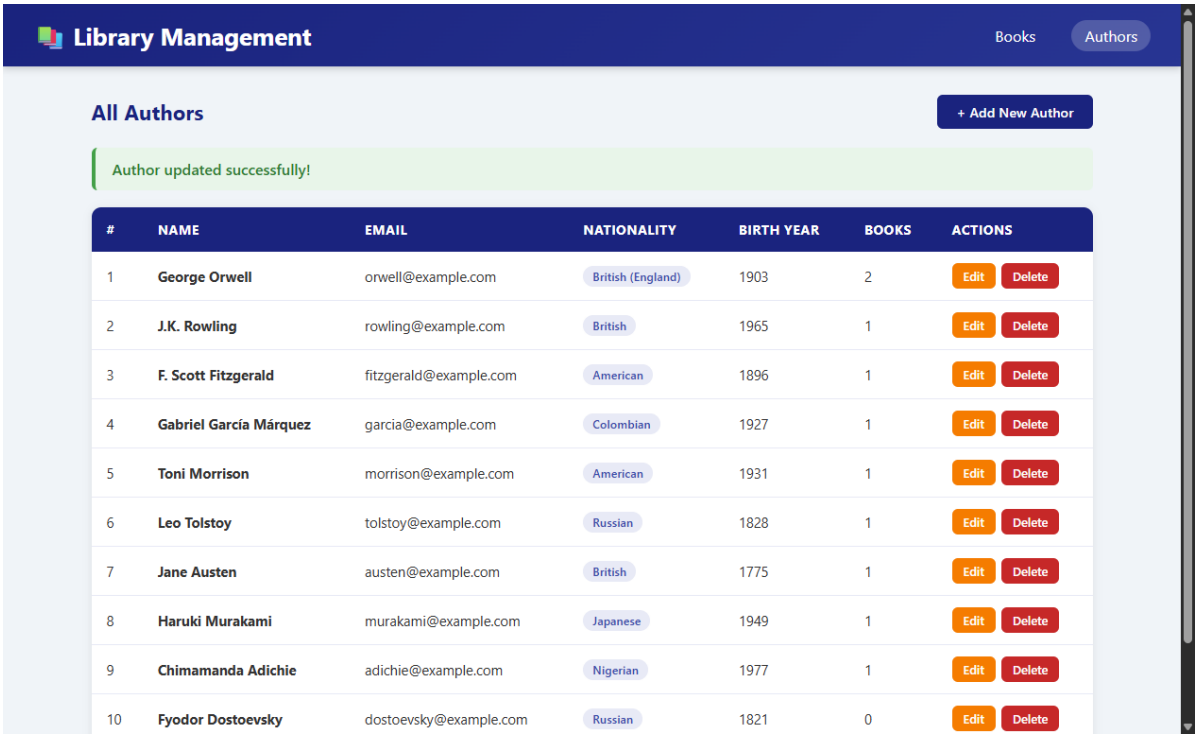


Fig 15: Edit Author form — nationality modified



#	NAME	EMAIL	NATIONALITY	BIRTH YEAR	BOOKS	ACTIONS
1	George Orwell	orwell@example.com	British (England)	1903	2	Edit Delete
2	J.K. Rowling	rowling@example.com	British	1965	1	Edit Delete
3	F. Scott Fitzgerald	fitzgerald@example.com	American	1896	1	Edit Delete
4	Gabriel García Márquez	garcia@example.com	Colombian	1927	1	Edit Delete
5	Toni Morrison	morrison@example.com	American	1931	1	Edit Delete
6	Leo Tolstoy	tolstoy@example.com	Russian	1828	1	Edit Delete
7	Jane Austen	austen@example.com	British	1775	1	Edit Delete
8	Haruki Murakami	murakami@example.com	Japanese	1949	1	Edit Delete
9	Chimamanda Adichie	adichie@example.com	Nigerian	1977	1	Edit Delete
10	Fyodor Dostoevsky	dostoevsky@example.com	Russian	1821	0	Edit Delete

Fig 16: Authors list after successful update

7. Application Architecture

Layer	Class / Interface	Responsibility
Entity	Author, Book	JPA-mapped domain objects with validation
Repository	AuthorRepository, BookRepository	DB access; extends JpaRepository; custom JPQL
DTO	BookWithAuthorDTO	Carries inner-join projection to view layer
Service	AuthorServiceImpl, BookServiceImpl	Business logic, duplicate checks, transactions
Controller	AuthorController, BookController	HTTP request handling, model binding, redirects
Exception	GlobalExceptionHandler (@ControllerAdvice)	Catches unhandled exceptions, renders error page
View	6 JSP pages + error.jsp	HTML rendering with JSTL / Spring form tags
Config / Init	DataInitializer, application.properties	Seed data, H2 config, JSP view resolver

8. Testing

The project includes **31 unit tests** covering all repository and service methods, all of which pass. Repository tests use @DataJpaTest with an in-memory H2 database. Service tests use **Mockito** to mock the repository layer.

Test Class	Tests	Focus
AuthorRepositoryTest	7	@DataJpaTest — findByEmail, findByNationality, findAuthorsWithBooks, CRU
BookRepositoryTest	7	@DataJpaTest — findByIsbn, findByGenre, findAllBooksWithAuthorDetails (in
AuthorServiceTest	8	Mockito — save duplicate check, update, delete, getAll
BookServiceTest	9	Mockito — save duplicate ISBN, update, getAllBooksWithAuthorDetails, delet
TOTAL	**31**	**All passing ✓**

Sample Service Test

```
@Test
void saveBook_shouldThrow_whenIsbnAlreadyExists() {
when(bookRepository.findByIsbn(anyString())).thenReturn(Optional.of(book));

assertThatThrownBy(() -> bookService.saveBook(book))
.isInstanceOf(DataIntegrityViolationException.class)
.hasMessageContaining("already exists");

verify(bookRepository, never()).save(any());
}
```

Sample Repository Test

```
@Test
void findAllBooksWithAuthorDetails_shouldReturnInnerJoinResult() {
List<BookWithAuthorDTO> result = bookRepository.findAllBooksWithAuthorDetails();
assertThat(result).hasSize(1);
BookWithAuthorDTO dto = result.get(0);
assertThat(dto.getBookTitle()).isEqualTo("Test Book");
assertThat(dto.getAuthorName()).isEqualTo("Jane Doe");
assertThat(dto.getAuthorNationality()).isEqualTo("American");
}
```

9. H2 Database Console

The H2 web console is enabled at **http://localhost:8080/h2-console** for inspecting the live database. Connection details: JDBC URL = jdbc:h2:mem:librarydb, Username = sa, Password = (empty).

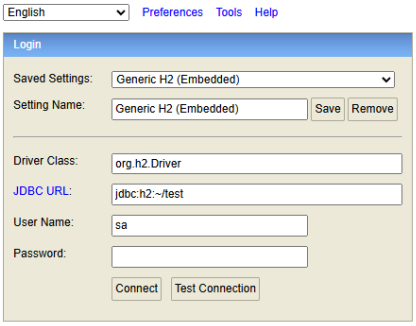


Fig 17: H2 database console

10. Challenges and Solutions

• JSP rendering with Spring Boot

Spring Boot's embedded Tomcat does not compile JSPs by default — a common pitfall. Adding tomcat-embed-jasper with scope provided and packaging the project as WAR resolved this.

• Inner join returning DTOs via JPQL

Returning a DTO (not an entity) from a JPQL query requires the constructor-expression syntax SELECT new com.library.dto.BookWithAuthorDTO(...). Every constructor parameter must exactly match the DTO constructor signature — a mismatch causes a runtime exception that is easy to miss.

• Handling duplicate key violations gracefully

Spring wraps JDBC constraint exceptions inside DataIntegrityViolationException. Rather than letting the global handler render a generic error page, the service layer detects duplicates *before* hitting the database and throws the exception with a descriptive message that is then bound back to the form field via BindingResult.rejectValue().

• Update preserving the author relationship

When updating a Book, Hibernate needs the full Author object, not just an ID. The controller accepts the authorId as a separate request parameter, looks up the Author entity, and sets it on the Book before calling the service — avoiding a TransientPropertyValueException.

• @DataJpaTest isolating the DataInitializer

The DataInitializer (CommandLineRunner) runs on every test context start and pollutes test data. Tests call deleteAll() in @BeforeEach to start with a clean slate, and each test inserts only the data it needs.

11. Project Structure

```
src/
■■■ main/
■ ■■■ java/com/library/
■ ■ ■■■ LibraryApplication.java # Spring Boot entry point
■ ■ ■■■ DataInitializer.java # Seeds 10 authors + 10 books
■ ■ ■■■ entity/
■ ■ ■ ■■■ Author.java # @Entity — One-to-Many
■ ■ ■ ■■■ Book.java # @Entity — Many-to-One
■ ■ ■■■ dto/
■ ■ ■ ■■■ BookWithAuthorDTO.java # Inner-join projection
■ ■ ■■■ repository/
■ ■ ■ ■■■ AuthorRepository.java # JpaRepository + custom JPQL
■ ■ ■ ■■■ BookRepository.java # JpaRepository + inner join query
■ ■ ■■■ service/
■ ■ ■ ■■■ AuthorService.java # Interface
■ ■ ■ ■■■ AuthorServiceImpl.java # Business logic + duplicate checks
■ ■ ■ ■■■ BookService.java # Interface
■ ■ ■ ■■■ BookServiceImpl.java # Business logic + duplicate checks
■ ■ ■■■ controller/
■ ■ ■ ■■■ AuthorController.java # CRUD request handling
■ ■ ■ ■■■ BookController.java # CRUD request handling
■ ■ ■ ■■■ HomeController.java # Redirects / to /books
■ ■ ■■■ exception/
■ ■ ■■■ GlobalExceptionHandler.java # @ControllerAdvice
■ ■■■ resources/
■ ■ ■■■ application.properties # H2 config, JSP resolver
■ ■■■ webapp/WEB-INF/views/
■ ■■■ books/list.jsp, add.jsp, edit.jsp
■ ■■■ authors/list.jsp, add.jsp, edit.jsp
■ ■■■ error.jsp
■■■ test/java/com/library/
■■■ repository/AuthorRepositoryTest.java # 7 tests (@DataJpaTest)
■■■ repository/BookRepositoryTest.java # 7 tests (@DataJpaTest)
■■■ service/AuthorServiceTest.java # 8 tests (Mockito)
■■■ service/BookServiceTest.java # 9 tests (Mockito)
```

12. How to Run the Application

Prerequisites:
Java 11+ installed (JDK). No Maven installation needed — the project includes the Maven wrapper.

Build:
Open a terminal in the project root and run: mvnw.cmd clean package

Run:
Execute: mvnw.cmd spring-boot:run

Access:
Open **http://localhost:8080** in a browser — redirects to the Books list.

H2 Console:
Browse to **http://localhost:8080/h2-console**, JDBC URL: jdbc:h2:mem:librarydb, user: sa, password: (empty).

Run Tests:
Execute: mvnw.cmd test — all 31 tests should pass.

13. GitHub Repository

The complete source code for this project is available on GitHub. The repository contains all source files, the Maven wrapper, and this document.

GitHub URL	`https://github.com/<your-username>/library-management`

End of Document